

IITM COLLEGE OF ENGINEERING



Data and File Structures (Lab)

Course code: 23BCA402DS02

BCA 2nd Semester
(2024-2027)

Submitted to: Mr. Anurag Patel

Submitted by: Sandeep Shukla

Registration No: 2413071273

INDEX

S.No	List of Experiments	Page No.	Date of Experiment	Faculty Signature
1.	Program to demonstrate basic data structure operations such as creating and displaying elements in an array.			
2.	Program to insert an element at a specified position in a linear array.			
3.	Program to delete an element at a specified position in a linear array.			
4.	Program to implement and demonstrate a sequential search on an array.			
5.	Program to implement and demonstrate the Bubble sort algorithm.			
6.	Program to implement and demonstrate a binary search on a sorted array.			
7.	Program to implement and demonstrate the Selection sort algorithm.			
8.	Program to implement and demonstrate the Insertion sort algorithm.			
9.	Program to implement and demonstrate the Quick sort algorithm.			
10.	Program to implement and demonstrate the Merge sort algorithm.			
11.	Program to implement a stack using an array and perform basic stack operations: push, pop, and display.			
12.	Program to implement a queue using an array and perform basic queue operations: enqueue, dequeue, and display.			
13.	Write a C program to implement a singly linked list and perform insertion, deletion, and traversal operations.			
14.	Write a C program to implement a binary tree and perform in-order, pre-order, and post-order traversal using recursion.			
15.	Write a C program to implement a binary search tree (BST) and perform insertion, deletion, and search operations.			
16.	Write a C program to represent a graph using an adjacency matrix and perform a depth first search (DFS).			

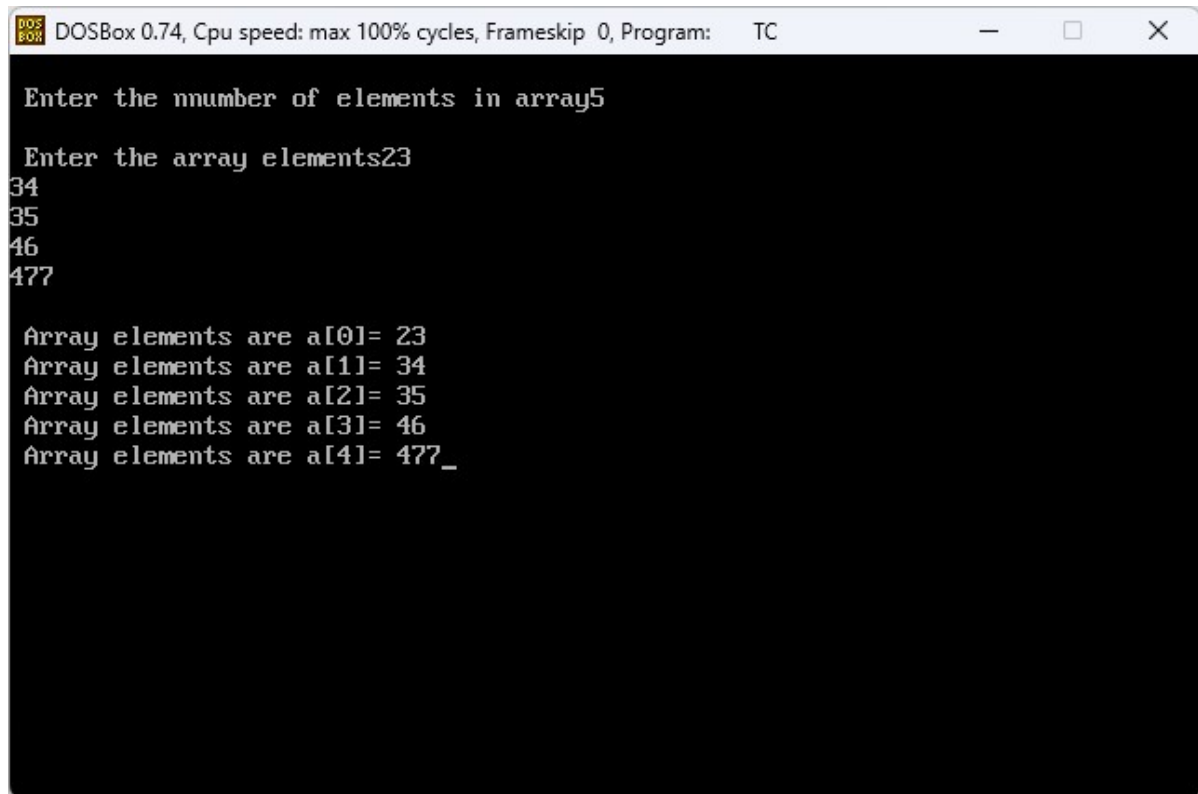
17.	Write a C program to represent a graph using an adjacency list and perform a breadth-first search (BFS).			
18.	Write a C program to demonstrate basic file operations: open, read, write, and close.			

PROGRAM

1. Write a C program to demonstrate basic data structure operations such as creating and displaying elements in an array

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[20],i,num;
    clrscr();
    printf("\n Enter the number of elements in array");
    scanf("%d",&num);
    printf("\n Enter the array elements");
    for(i=0;i<num;i++)
    {
        scanf("%d",&a[i]);
    }
    for(i=0;i<num;i++)
    {
        printf("\n Array elements are a[%d]= %d",i,a[i]);
    }
    getch();
}
```

OUTPUT:-



The screenshot shows a DOSBox window with the title bar "DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC". The window contains the following text:

```
Enter the number of elements in array5
Enter the array elements23
34
35
46
477

Array elements are a[0]= 23
Array elements are a[1]= 34
Array elements are a[2]= 35
Array elements are a[3]= 46
Array elements are a[4]= 477_
```

PROGRAM

2. Write a C program to insert an element at a specified position in a linear array.

```
#include <stdio.h>

#include <conio.h>

void main ()
{
    /*****Array creation and display*****/
    int i,a[20],n;

    clrscr();

    printf("\n Enter the numbers of elements in array");
    scanf("%d",&n);

    printf ("\n Enter the array elements");
    for (i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }

    printf("\n Display the elements in a[%d] array",n);
    for(i=0;i<n;i++)
    {
        printf("\n a[%d] = %d",i,a[i]);
    }

    /*** Inserting an element at a specified position in array*****/
    int pos,data;

    printf("\n enter the position to insert ");
    scanf("%d",&pos);

    printf("\n enter the data to enter");
```

```

scanf("%d",&data);
for (i=n-1;i>=pos-1;i--)
{
a[i+1]=a[i];
}
a[pos-1]=data;
n=n+1;
printf("\n display the array with inserted values");
for(i=0;i<n;i++)
{
printf("\n a[%d]=%d",i,a[i]);
}
getch();
}

```

OUTPUT:-

```

Enter the numbers of elements in array5

```

```

Enter the elements2

```

```

3
4
5
6

```

```

Display the elements in a[5] array

```

```

a[0] = 2
a[1] = 3
a[2] = 4
a[3] = 5
a[4] = 6

```

```

Enter the position to insert 5

```

```

Enter the data to insert9

```

```

Display the array with inserted values

```

```

a[0]=2
a[1]=3
a[2]=4
a[3]=5
a[4]=9
a[5]=6_

```

PROGRAM

3. Write a C program to delete an element at a specified position in a linear array.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    /*****Array creation and display*****/
    int i,a[20],n;
    clrscr();
    printf("\n Enter the numbers of elements in array");
    scanf("%d",&n);
    printf ("\n Enter the array elements");
    for (i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("\n Display the elements in a[%d] array",n);
    for(i=0;i<n;i++)
    {
        printf("\n a[%d] = %d",i,a[i]);
    }
    /**** Delete an element at a specified position in array*****/ int
    pos1,temp;
    printf("\n Enter the position to delete : ");
    scanf("%d",&pos1);
    if (pos1>n)
    {
        printf("\n Position exceeds size of array - Invalid option");
    }
    else
    {
        temp =a[pos1-1];
        for (i=pos1-1;i<=n-1;i++)
        {
            a[i]=a[i+1];
        }
        n--;
    }
```



```
printf("\n After deletion of element in %d position",pos1);
for (i=0;i<n;i++)
{
printf("\n a[%d]=%d",i,a[i]);
}
printf("\n Deleted element is : %d",temp);
}
getch();
}
```

OUTPUT:-

```
Enter the numbers of elements in array5
Enter the elements2
3
4
5
6

Display the elements in a[5] array
a[0] = 2
a[1] = 3
a[2] = 4
a[3] = 5
a[4] = 6
Enter the position to delete : 3

After deletion of element in 3 position
a[0]=2
a[1]=3
a[2]=5
a[3]=6
Deleted element is : 4
```

PROGRAM

4. Write a C program to implement and demonstrate a sequential search on an array

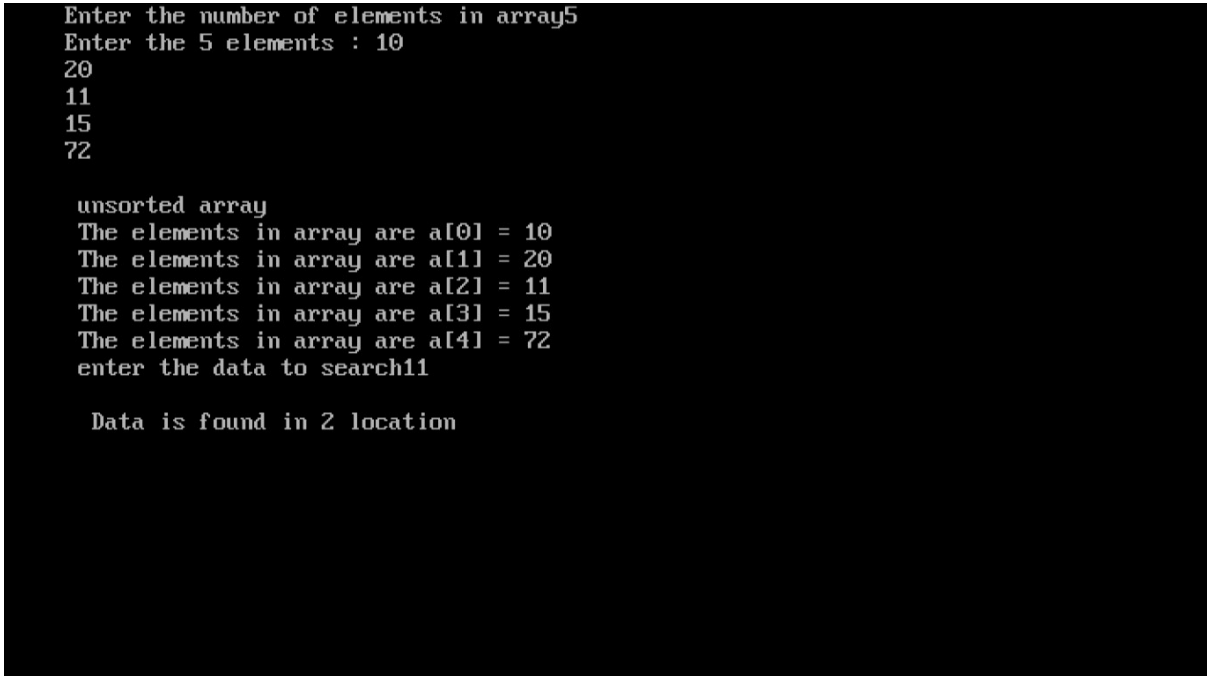
```
#include <stdio.h>
#include <conio.h>
void printarray(int *a,int n)
{
    int i;
    for(i=0;i<n;i++)
    {
        printf("\n The elements in array are a[%d] = %d",i,a[i]);
    }
}

void main()
{
    int a[10],i,n,data,match;
    clrscr();
    printf("Enter the number of elements in array");
    scanf("%d",&n);
    printf("Enter the %d elements : ",n);
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    printf ("\n unsorted array");
    printarray (a,n);

    //*****Linear Search*****/
    printf("\n enter the data to search");
    scanf("%d",&data);
    match =0;
    for (i=0;i<=n;i++)
    {
        if (a[i]==data)
        {
            printf("\n Data is found in %d location ",i);
```

```
match =1;
}}
if (match==0)
{
printf(" Data not found");
}
getch();
}
```

OUTPUT:-



```
Enter the number of elements in array5
Enter the 5 elements : 10
20
11
15
72

unsorted array
The elements in array are a[0] = 10
The elements in array are a[1] = 20
The elements in array are a[2] = 11
The elements in array are a[3] = 15
The elements in array are a[4] = 72
enter the data to search11

Data is found in 2 location
```

PROGRAM

5. Write a C program to implement and demonstrate bubble sort algorithm

```
#include <stdio.h>
#include <conio.h>
void printarray(int *a,int n)
{
    int i;
    for(i=0;i<n;i++)
    {
        printf("\n elements in array are a[%d]= %d ",i,a[i]);
    }
}

void main()
{
    int i,j,a[]={12,1,15,2,0};
    int temp,n;
    clrscr();
    // size of one element is 4 bytes , size of a[] = 16 bytes
    n=sizeof(a)/sizeof(a[0]);
    printf ("size of array is : %d",n);
    printf ("\n unsorted array");
    printarray(a,n);
    /***** Ascending order sorting using bubble sort*****/ for (j=0;j<n-
1;j++)
    {
        for (i=0;i<n-1-j;i++)
        {
            if (a[i]>a[i+1])
            {
                temp=a[i];
                a[i]=a[i+1];
                a[i+1]=temp;
            }
        }
        printf ("\n pass %d",j);
        printarray(a,n);
    }
    //printf("\n Sorted array in Ascending order");
```

```
//printarray(a,n);  
getch();  
}
```

OUTPUT:-

```
size of array is : 5  
unsorted array  
elements in array are a[0]= 12  
elements in array are a[1]= 1  
elements in array are a[2]= 15  
elements in array are a[3]= 2  
elements in array are a[4]= 0 _
```

```
elements in array are a[4]= 0
pass 0
elements in array are a[0]= 1
elements in array are a[1]= 12
elements in array are a[2]= 2
elements in array are a[3]= 0
elements in array are a[4]= 15
pass 1
elements in array are a[0]= 1
elements in array are a[1]= 2
elements in array are a[2]= 0
elements in array are a[3]= 12
elements in array are a[4]= 15
pass 2
elements in array are a[0]= 1
elements in array are a[1]= 0
elements in array are a[2]= 2
elements in array are a[3]= 12
elements in array are a[4]= 15
pass 3
elements in array are a[0]= 0
elements in array are a[1]= 1
elements in array are a[2]= 2
elements in array are a[3]= 12
elements in array are a[4]= 15
```

PROGRAM

6. Write a C program to implement and demonstrate a binary search on a sorted array.

```
#include <stdio.h>
#include <conio.h>
void printarray (int *a,int n)
{
    int i;
    for (i=0;i<n;i++)
    {
        printf("%4d",a[i]);
    }
}

int binarysearch(int a[],int low,int high,int data)
{
    int mid;
```

```

while (low<=high)
{
mid=(low+high)/2;
if(a[mid]==data)
{
return mid;
}
if(a[mid]<data)
{
low=mid+1;
}
else
{
high=mid-1;
}}
return -1;
}

void main()
{
int a[] = {2,5,8,12,16,23,38,56,72,91,100};
//int a[]={2};
int n,data,result;
n = sizeof(a)/sizeof(a[0]);
clrscr();
printarray(a,n);
printf(" \n The size of array is %d", n);
printf("\n Enter the data to search: ");
scanf("%d",&data);
result = binarysearch (a,0,n-1,data);
if (result== -1)
{
printf("\n\n Element is not present in array"); }
else
{
printf("\n\n Element is present at index %d",result); }
getch();
}

```

OUTPUT:-

```
2 5 8 12 16 23 38 56 72 91 100
The size of array is 11
Enter the data to search: 56
```

```
Element is present at index 7
```

PROGRAM

7. Write a C program to implement and demonstrate the Selection sort algorithm.

```
#include <stdio.h>
#include <conio.h>
void printarray(int *a,int n)
{
    int i;
    for(i=0;i<n;i++)
```



```

{
printf("%4d",a[i]);
}}

void selectionsort(int *a,int size)
{
int i,j,k,imin;
int start;
start = a[0];
for(i=0;i<size-1;i++)
{
imin=i; // get the index of minimum data
for(j=i+1;j<size;j++)
if(a[j]<a[imin])
imin=j;
// placing in correct position
int temp;
temp=a[i];
a[i]=a[imin];
a[imin]=temp;
printf("\nLoop %d: ",i);
for(k=0;k<size;k++)
{
printf(" %4d",a[k]);
}}}

void main()
{
int n,i;
int a[]={12,19,55,2};
clrscr();
n=sizeof(a)/sizeof(a[0]);
printf("\n the size of array is : %d",n);
printf ("\n Array before sorting: ");
printarray(a,n);
selectionsort(a,n);
printf("\n Array after sorting");
printarray(a,n);
getch();
}

```

OUTPUT:-

```
the size of array is : 6
Array before sorting: 12 19 55 2 77 1
Loop 0: 1 19 55 2 77 12
Loop 1: 1 2 55 19 77 12
Loop 2: 1 2 12 19 77 55
Loop 3: 1 2 12 19 77 55
Loop 4: 1 2 12 19 55 77
Array after sorting 1 2 12 19 55 77_
```

PROGRAM

8. Write a C program to implement and demonstrate the Insertion sort algorithm.

```
#include <stdio.h>
#include <conio.h>
```

```

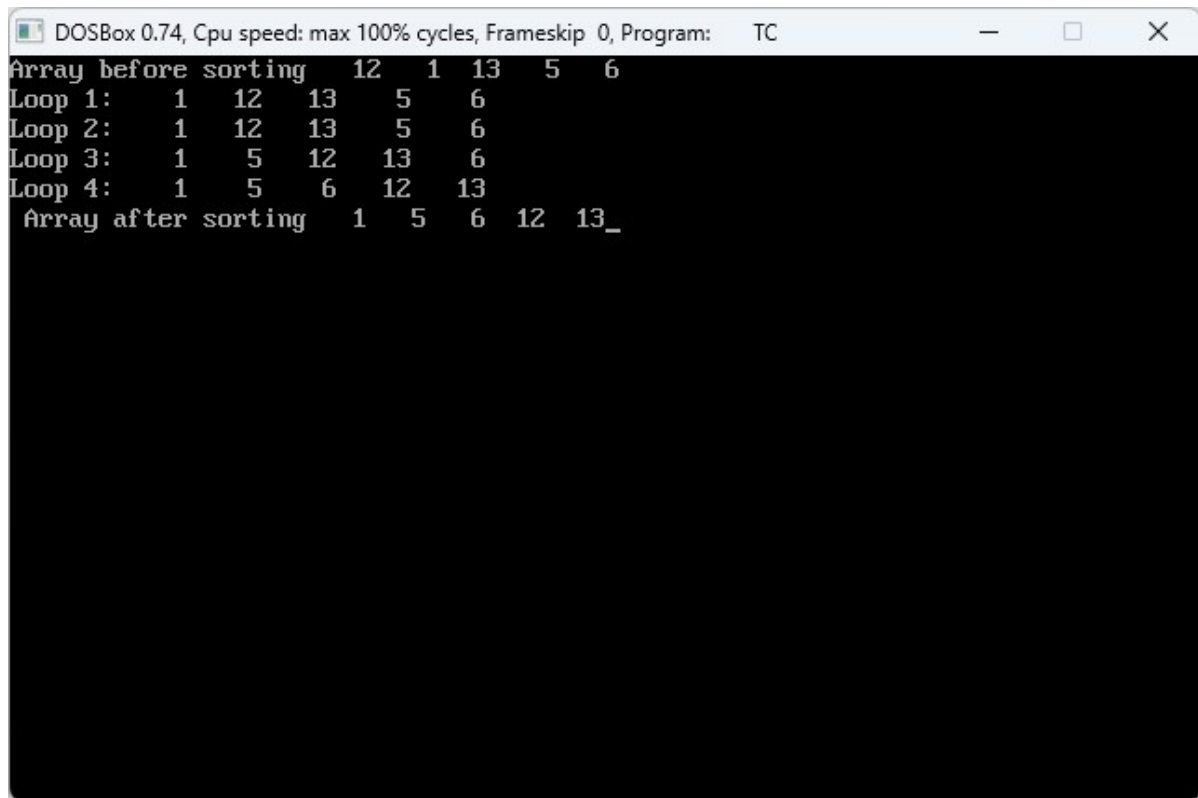
void insertionsort(int a[],int n)
{
    int i,j,k,key;
    for(i=1;i<n;i++)
    {
        key=a[i];
        j=i-1;
        //Move elements greater then key to one position ahead of their current pos
        while(j>=0 && a[j]>key)
        {
            a[j+1]=a[j];
            j=j-1;
        }
        a[j+1]=key;
        printf("\nLoop %d:",i);
        for(k=0;k<n;k++)
        {
            printf(" %4d",a[k]);
        }
    }

    void printarray(int a[],int n)
    {
        int i;
        for(i=0;i<n;i++)
        {
            printf("%4d",a[i]);
        }

        void main()
        {
            int a[]={12,71,13,5,6};
            int n;
            clrscr();
            n=sizeof(a)/sizeof(a[0]);
            printf("Array before sorting ");
            printarray(a,n);
            insertionsort(a,n);
            printf("\n Array after sorting");
            printarray(a,n);
            getch();
        }

```

OUTPUT:-



```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Array before sorting 12 1 13 5 6
Loop 1: 1 12 13 5 6
Loop 2: 1 12 13 5 6
Loop 3: 1 5 12 13 6
Loop 4: 1 5 6 12 13
Array after sorting 1 5 6 12 13_
```

PROGRAM

9. Write a C program to implement and demonstrate the Quick sort algorithm.

```
#include <stdio.h>
```

```

#include <conio.h>
int partition(int arr[],int st,int end)
{
    int temp,idx=st-1,pivot=arr[end];
    for(int j=st;j<end;j++)
    {
        if(arr[j]<=pivot)
        {
            idx++;
            temp = arr[j];
            arr[j]=arr[idx];
            arr[idx]=temp;
        }
    }
    idx++;
    temp=arr[idx];
    arr[idx]=arr[end];
    arr[end]=temp;
    return idx;
}

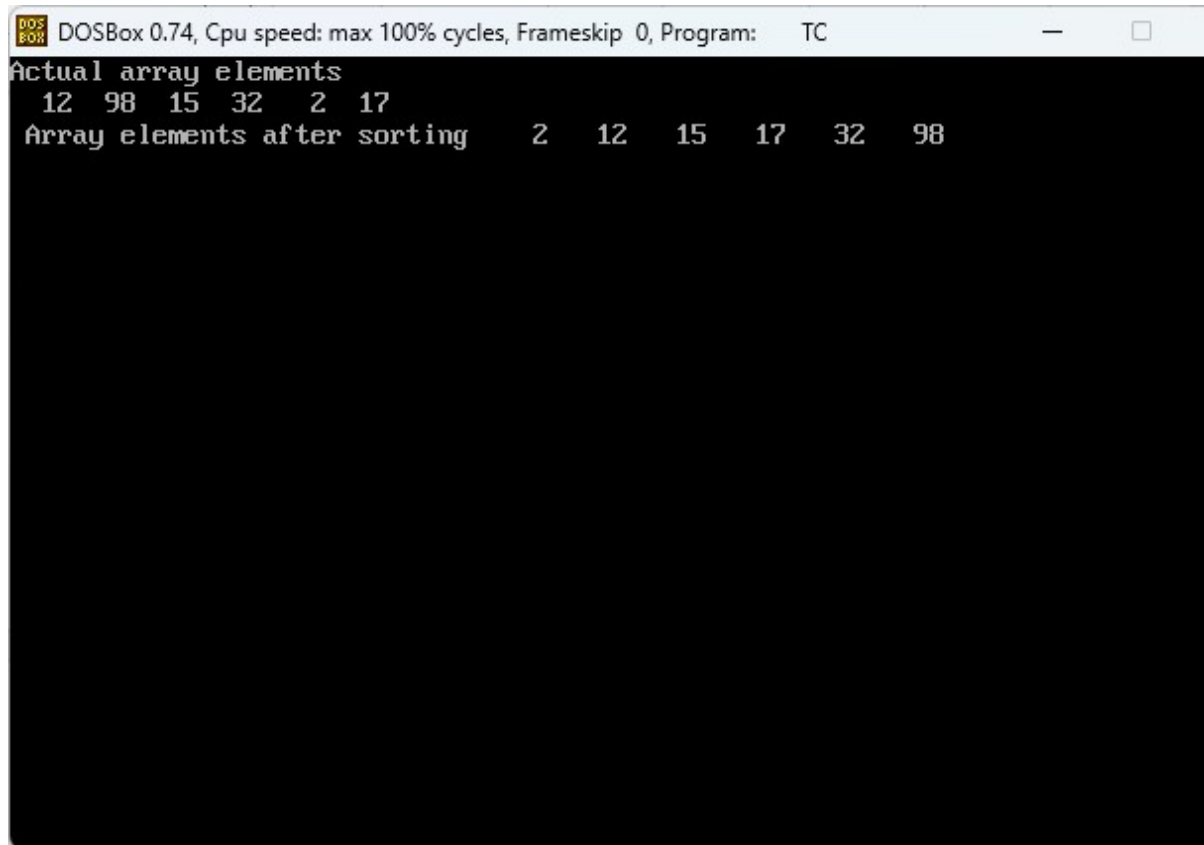
void quicksort(int arr[],int st,int end)
{
    int size;
    if(st<end)
    {
        int pivIdx =partition (arr,st,end);
        quicksort(arr,st,pivIdx-1);//left half
        quicksort(arr,pivIdx+1,end);//right half
    }
}

void main()
{
    int arr[]={12,98,15,32,2,17};
    int size=sizeof(arr)/sizeof(arr[0]);
    clrscr();
    printf("Actual array elements\n");
    for (int i=0;i<size;i++)
    {
        printf("%4d",arr[i]);
    }
    quicksort(arr,0,size-1);
    printf ("\n Array elements after sorting");
    for(i=0;i<size;i++)
    {

```

```
printf(" %4d",arr[i]); }  
getch();  
}
```

OUTPUT:-



```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC  
Actual array elements  
12 98 15 32 2 17  
Array elements after sorting 2 12 15 17 32 98
```

PROGRAM

10. Write a C program to implement and demonstrate the Merge sort algorithm.

```
#include <stdio.h>
```

```

#include <conio.h>
#define MAX_SIZE 100
void merge(int arr[],int st,int mid,int end)
{
    int leftarr[MAX_SIZE],rightarr[MAX_SIZE];
    int n1=(mid-st)+1;
    int n2=end-mid;
    //create temporary array
    //int leftarr[n1], rightarr[n2];

    int i,j,k;
    //copy data to temporary arrays
    for(i=0;i<n1;i++)
    {
        leftarr[i]=arr[st+i];
    }
    for(j=0;j<n2;j++)
    {
        rightarr[j]=arr[mid+1+j];
    }
    //merge the temp array into arr
    i=0; // initial index of left sub array
    j=0; // initial index of right sub array
    k=st; // initial index of merged subarray
    while(i<n1 && j<n2)
    {
        if (leftarr[i]<=rightarr[j])
        {
            arr[k]=leftarr[i];
            i++;
        }
        else
        {
            arr[k]=rightarr[j];
            j++;
        }
        k++;
    }
    //copy the remaining elements of leftarr[]
    while(i<n1)
    {
        arr[k]=leftarr[i];
        i++;
        k++;
    }
}

```

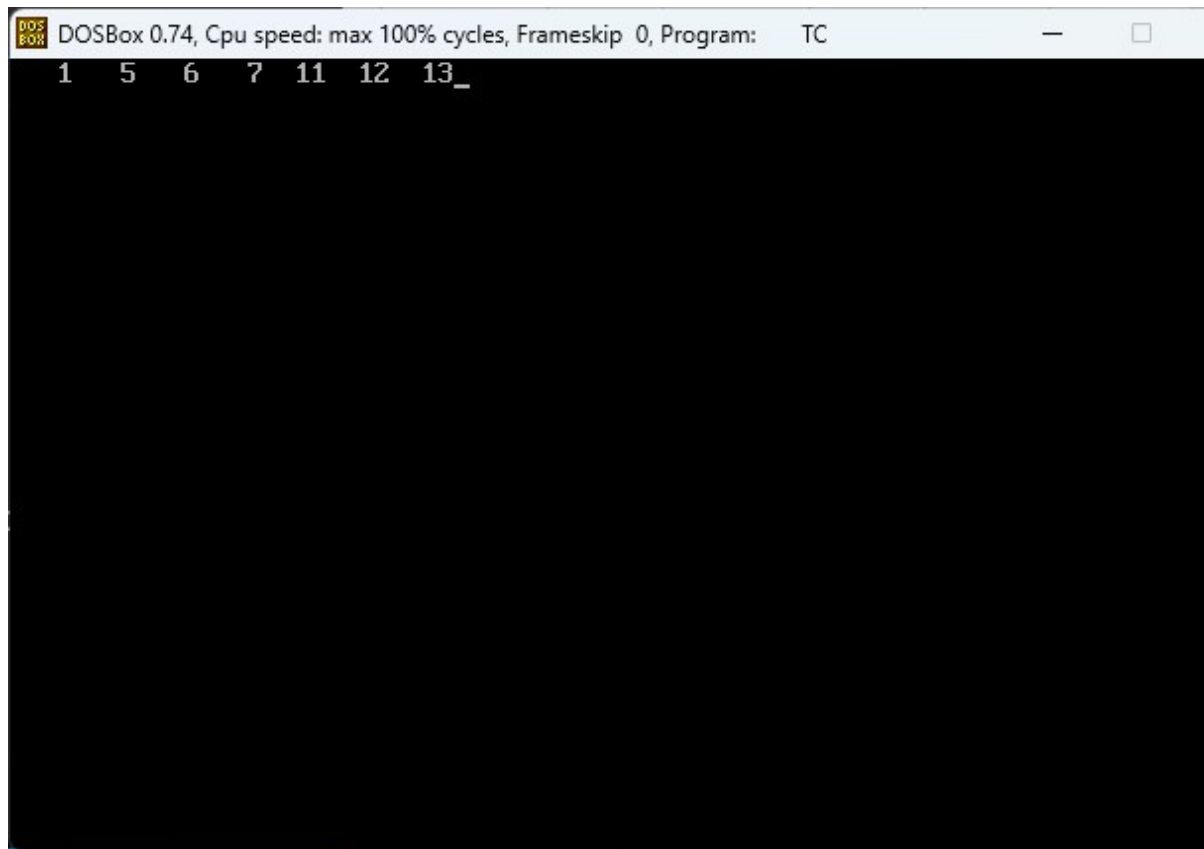
```

//copy the remaining elements of rightarr[]
while (j<n2)
{
arr[k]=rightarr[j];
j++;
k++;
}
}
// the subarray to be sorted is in the index range [left - right] void
mergesort(int arr[],int st,int end)
{
if(st<end)
{
//calculate mipoint
int mid=st+(end-st)/2;
//sort first and second halves
mergesort(arr,st,mid);
mergesort(arr,mid+1,end);

//merge the sorted halves
merge(arr,st,mid,end);
}
}
void main()
{
int arr[] ={12, 11, 1, 13, 5, 6, 7};
int i,n;
clrscr();
n=sizeof(arr)/sizeof(arr[0]);
//sorting arr using mergesort
mergesort(arr,0,n-1);
for(i=0;i<n;i++)
{
printf("%4d",arr[i]);
}
getch();
}

```

OUTPUT:-



PROGRAM

11. Write a C program to implement a stack using an array and perform basic stack operations: push, pop, and display.

```
#include <stdio.h>
#define MAX 5 // Maximum size of the stack

int stack[MAX], top = -1;

// Function to push an element onto the stack
void push() {
    int value;
    if (top == MAX - 1) {
        printf("Stack Overflow! Cannot push more elements.\n");
    } else {
        printf("Enter the value to push: ");
        scanf("%d", &value);
        stack[++top] = value;
        printf("%d pushed into the stack.\n", value);
    }
}

// Function to pop an element from the stack
void pop() {
    if (top == -1) {
        printf("Stack Underflow! No elements to pop.\n");
    } else {
        printf("%d popped from the stack.\n", stack[top--]);
    }
}

// Function to display the stack
void display() {
    if (top == -1) {
        printf("Stack is empty.\n");
    } else {
        printf("Stack elements: ");
        for (int i = 0; i <= top; i++) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main() {
    int choice;
```

```
while (1) {
printf("\nStack Operations:\n");
printf("1. Push\n");
printf("2. Pop\n");
printf("3. Display\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
case 1: push(); break;
case 2: pop(); break;
case 3: display(); break;
case 4: printf("exit");
break;
default: printf("Invalid choice! Please try again.\n"); }
if (choice==4)
break;
}
return 0;
}
```

OUTPUT:-

PROGRAM

12. Write a C program to implement a queue using an array and perform basic queue operations: enqueue, dequeue, and display.

```
#include <stdio.h>
#define N 5

int queue[N];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == N - 1) {
        printf("Queue is full (Overflow)\n");
    } else {
        /* if (front == -1)
            front = 0;
            rear++;
            queue[rear] = value;
            printf("Enqueued %d\n", value); */
        if (front == -1 && rear == -1)
        {
            front=rear=0;
            queue[rear] = value;
        }
        else
        {
            rear++;
            queue[rear]=value;
            printf("Enqueued %d\n ", value);}
    }
}

void dequeue() {
    // if (front == -1 || front > rear) {
    if((front == -1 && rear == -1) || front > rear) {
        printf("Queue is empty (Underflow)\n");
    } else {
        printf("Dequeued %d\n", queue[front]);
        front++;
    }
}

void display() {
    // if (front == -1 || front > rear) {
    if((front == -1 && rear == -1) || front > rear) {
        printf("Queue is empty\n");
    }
```

```

    } else {
        printf("Queue elements: ");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}

int main() {
    int choice, value;

    while (1) {
        printf("\n--- Queue Menu ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter the value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting...\n");
                return 0;
            default:
                printf("Invalid choice! Try again.\n");
        }
    }

    // return 0;
}

```

OUTPUT:-

PROGRAM

13. Write a C program to implement a singly linked list and perform insertion, deletion and traversal operation.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define a Node structure
```

```
struct Node {
```

```
    int data;
```

```
    struct Node *next;
```

```
};
```

```
// Function to create a new node
```

```
struct Node* createNode(int data) {
```

```
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
```

```
    newNode->data = data;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to insert a node at the beginning
```

```
void insertAtBeginning(struct Node **head, int data) {
```

```
    struct Node *newNode = createNode(data);
```

```
    newNode->next = *head;
```

```
    *head = newNode;
```

```
}
```

```
// Function to insert a node at the end
```

```
void insertAtEnd(struct Node **head, int data) {
```

```

    struct Node *newNode = createNode(data);

    if (*head == NULL) {

        *head = newNode;

        return;

    }

    struct Node *temp = *head;

    while (temp->next != NULL) {

        temp = temp->next;

    }

    temp->next = newNode;
}

// Function to delete a node by key
void deleteNode(struct Node **head, int key) {

    struct Node *temp = *head, *prev = NULL;

    // If the head node itself holds the key
    if (temp != NULL && temp->data == key)

        *head = temp->next; // Change head

        free(temp); // Free the old head

        return;

    // Search for the key to be deleted
    while (temp != NULL && temp->data != key)

        prev = temp;

        temp = temp->next;

    }

    // If key was not found

```



```
if (temp == NULL) {  
    printf("Node with value %d not found.\n", key);  
    return;  
}
```

```
// Unlink the node from the linked list  
prev->next = temp->next;  
free(temp); // Free the memory of the node  
}
```

```
// Function to traverse the linked list and print its elements
```

```
void traverse(struct Node *head) {  
    struct Node *temp = head;  
    if (temp == NULL) {  
        printf("List is empty.\n");  
        return;  
    }  
    printf("Linked List: ");  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```

```
}
```

```
int main() {
```

```
    struct Node *head = NULL; // Initialize the linked list to be empty
```

```
    // Insert some nodes at the beginning and end
```

```
    insertAtBeginning(&head, 10);
```

```
    insertAtBeginning(&head, 20);
```

```
    insertAtEnd(&head, 30);
```

```
    insertAtEnd(&head, 40);
```

```
    // Print the linked list
```

```
    traverse(head);
```

```
    // Delete a node
```

```
    deleteNode(&head, 20);
```

```
    printf("After deleting node with value 20:\n");
```

```
    traverse(head);
```

```
    deleteNode(&head, 40);
```

```
    printf("After deleting node with value 40:\n");
```

```
    traverse(head);
```

```
    return 0;
```

OUTPUT:-

Output

Clear

Linked List: 20 -> 10 -> 30 -> 40 -> NULL

After deleting node with value 20:

Linked List: 10 -> 30 -> 40 -> NULL

After deleting node with value 40:

Linked List: 10 -> 30 -> NULL

=== Code Execution Successful ===

PROGRAM

14. Write a C program to implement a binary tree and perform in-order, pre-order and post-order traversal using recursion.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define a Node structure for the binary tree
```

```
struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
};
```

```
// Function to create a new node with given data
```

```
struct Node* createNode(int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}
```

```
// Pre-order traversal (Root -> Left -> Right)
```

```
void preOrderTraversal(struct Node* root) {  
    if (root != NULL) {  
        printf("%d ", root->data); // Visit the root  
        preOrderTraversal(root->left); // Traverse the left subtree  
        preOrderTraversal(root->right); // Traverse the right subtree  
    }  
}
```

```
// In-order traversal (Left -> Root -> Right)
void inOrderTraversal(struct Node* root) {
    if (root != NULL) {
        inOrderTraversal(root->left); // Traverse the left subtree
        printf("%d ", root->data); // Visit the root
        inOrderTraversal(root->right); // Traverse the right subtree
    }
}
```

```
// Post-order traversal (Left -> Right -> Root)
void postOrderTraversal(struct Node* root) {
    if (root != NULL) {
        postOrderTraversal(root->left); // Traverse the left subtree
        postOrderTraversal(root->right); // Traverse the right subtree
        printf("%d ", root->data); // Visit the root
    }
}
```

```
int main() {
    // Manually creating a binary tree for demonstration
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);
    root->right->left = createNode(6);
    root->right->right = createNode(7);
}
```

```
// Perform traversals
printf("Pre-order Traversal: ");
preOrderTraversal(root);
printf("\n");

printf("In-order Traversal: ");
inOrderTraversal(root);
printf("\n");

printf("Post-order Traversal: ");
postOrderTraversal(root);
printf("\n");

return 0;
}
```

OUTPUT:-

```
Output Clear
Pre-order Traversal: 1 2 4 5 3 6 7
In-order Traversal: 4 2 5 1 6 3 7
Post-order Traversal: 4 5 2 6 7 3 1

=== Code Execution Successful ===
```

PROGRAM

15 Write a C program to implement a binary search tree and perform insertion, deletion and search operations.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Definition of the Node structure
```

```
struct Node {
```

```
    int data;
```

```
    struct Node *left, *right;
```

```
};
```

```
// Function to create a new node
```

```
struct Node* createNode(int data) {
```

```
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
```

```
    newNode->data = data;
```

```
    newNode->left = newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to insert a node in the Binary Search Tree
```

```
struct Node* insert(struct Node* root, int data) {
```

```
    if (root == NULL) {
```

```
        return createNode(data);
```

```
    }
```

```
// If the data is less than the root, insert in the left subtree
```

```
if (data < root->data) {
```

```
    root->left = insert(root->left, data);
```

```

    }

    // If the data is greater than the root, insert in the right subtree
    else if (data > root->data) {
        root->right = insert(root->right, data);
    }

    // Return the unchanged root pointer
    return root;
}

// Function to search for a node in the BST
struct Node* search(struct Node* root, int data) {
    // Base case: root is NULL or data is present at the root
    if (root == NULL || root->data == data) {
        return root;
    }

    // Data is smaller than root's data, so search in the left subtree
    if (data < root->data) {
        return search(root->left, data);
    }

    // Data is greater than root's data, so search in the right subtree
    return search(root->right, data);
}

// Function to find the minimum value node in a BST
struct Node* findMin(struct Node* root) {
    while (root->left != NULL) {

```



```
    root = root->left;
}
return root;
}
```

// Function to delete a node from the BST

```
struct Node* delete(struct Node* root, int data) {
```

```
    // Base case: If the tree is empty
```

```
    if (root == NULL) {
```

```
        return root;
```

```
    }
```

```
    // Recursive case: find the node to delete
```

```
    if (data < root->data) {
```

```
        root->left = delete(root->left, data); // Delete in the left subtree
```

```
    } else if (data > root->data) {
```

```
        root->right = delete(root->right, data); // Delete in the right subtree
```

```
    } else {
```

```
        // Node to be deleted is found
```

```
        // Case 1: Node has no children (leaf node)
```

```
        if (root->left == NULL && root->right == NULL) {
```

```
            free(root);
```

```
            root = NULL;
```

```
        }
```

```
        // Case 2: Node has one child
```

```
        else if (root->left == NULL) {
```

```
            struct Node* temp = root;
```

```
            root = root->right;
```

```

        free(temp);
    } else if (root->right == NULL) {
        struct Node* temp = root;
        root = root->left;
        free(temp);
    }
    // Case 3: Node has two children
    else {
        struct Node* temp = findMin(root->right); // Find the minimum node in the right
        subtree
        root->data = temp->data; // Copy the minimum value to the current node
        root->right = delete(root->right, temp->data); // Delete the duplicate node
    }
}

return root;
}

// Function to perform in-order traversal (Left -> Root -> Right)
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left); // Traverse left subtree
        printf("%d ", root->data); // Visit the root
        inorder(root->right); // Traverse right subtree
    }
}

int main() {
    struct Node* root = NULL;

```

```
// Insert nodes into the binary search tree
root = insert(root, 50);
root = insert(root, 30);
root = insert(root, 20);
root = insert(root, 40);
root = insert(root, 70);
root = insert(root, 60);
root = insert(root, 80);

printf("In-order Traversal of the BST: ");
inorder(root);
printf("\n");

// Search for a value
int searchVal = 40;
struct Node* result = search(root, searchVal);
if (result != NULL) {
    printf("Node with value %d found in the BST.\n", searchVal);
} else {
    printf("Node with value %d not found in the BST.\n", searchVal);
}

// Delete a node
int deleteVal = 20;
root = delete(root, deleteVal);
printf("In-order Traversal after deleting node with value %d: ", deleteVal);
inorder(root);
printf("\n");
```

```
// Delete another node
deleteVal = 30;
root = delete(root, deleteVal);
printf("In-order Traversal after deleting node with value %d: ", deleteVal);
inorder(root);
printf("\n");

return 0;
}
```

OUTPUT:-

Output

Clear

```
In-order Traversal of the BST: 20 30 40 50 60 70 80
Node with value 40 found in the BST.
In-order Traversal after deleting node with value 20: 30 40 50 60 70 80
In-order Traversal after deleting node with value 30: 40 50 60 70 80

=== Code Execution Successful ===
```

PROGRAM

16 Write a C program to represent a graph using an adjacency matrix and perform a depth-first search(DFS).

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define the maximum number of vertices in the graph
```

```
#define MAX_VERTICES 10
```

```
// Function to perform Depth-First Search (DFS)
```

```
void dfs(int graph[MAX_VERTICES][MAX_VERTICES], int visited[MAX_VERTICES], int vertex, int n) {
```

```
    // Mark the current vertex as visited and print it
```

```
    visited[vertex] = 1;
```

```
    printf("%d ", vertex);
```

```
    // Recur for all the vertices adjacent to this vertex
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (graph[vertex][i] == 1 && !visited[i]) {
```

```
            dfs(graph, visited, i, n);
```

```
        }
```

```
    }
```

```
}
```

```
// Function to initialize the graph and perform DFS
```

```
int main() {
```

```
    int graph[MAX_VERTICES][MAX_VERTICES];
```

```
    int visited[MAX_VERTICES] = {0}; // Array to keep track of visited vertices
```

```
    int n, e, u, v;
```

```
// Take the number of vertices and edges as input
printf("Enter the number of vertices: ");
scanf("%d", &n);

printf("Enter the number of edges: ");
scanf("%d", &e);

// Initialize the adjacency matrix with 0 (no edges)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        graph[i][j] = 0;
    }
}

// Input edges
printf("Enter the edges (u v) where u and v are vertices connected by an edge:\n");
for (int i = 0; i < e; i++) {
    scanf("%d %d", &u, &v);
    // Since it's an undirected graph, mark both u->v and v->u
    graph[u][v] = 1;
    graph[v][u] = 1;
}

// Perform DFS starting from vertex 0 (or any vertex)
printf("DFS starting from vertex 0:\n");
dfs(graph, visited, 0, n);

return 0;
```

```
}
```

OUTPUT:-

```
Output Clear  
Enter the number of vertices: 5  
Enter the number of edges: 4  
Enter the edges (u v) where u and v are vertices connected by an edge:  
0 1  
0 2  
1 3  
1 4  
DFS starting from vertex 0:  
0 1 3 4 2  
  
=== Code Execution Successful ===
```

PROGRAM

17 Write a C program to represent a graph using a adjacency list and perform a breadth-first search(BFS).

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define the maximum number of vertices
```

```
#define MAX_VERTICES 10
```

```
// Define a structure to represent an adjacency list node
```

```
struct Node {
```

```
    int vertex;
```

```
    struct Node* next;
```

```
};
```

```
// Define a structure to represent an adjacency list
```

```
struct Graph {
```

```
    struct Node* adjList[MAX_VERTICES]; // Array of adjacency lists
```

```
};
```

```
// Function to create a new adjacency list node
```

```
struct Node* createNode(int vertex) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->vertex = vertex;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to initialize the graph
```



```

void initGraph(struct Graph* graph, int n) {
    for (int i = 0; i < n; i++) {
        graph->adjList[i] = NULL; // Initialize each adjacency list as empty
    }
}

// Function to add an edge to the graph
void addEdge(struct Graph* graph, int u, int v) {
    // Add an edge from u to v
    struct Node* newNode = createNode(v);
    newNode->next = graph->adjList[u];
    graph->adjList[u] = newNode;

    // Add an edge from v to u (since the graph is undirected)
    newNode = createNode(u);
    newNode->next = graph->adjList[v];
    graph->adjList[v] = newNode;
}

// Function to perform BFS
void bfs(struct Graph* graph, int start, int n) {
    int visited[MAX_VERTICES] = {0}; // Array to keep track of visited vertices
    int queue[MAX_VERTICES]; // Queue for BFS
    int front = 0, rear = 0;

    // Mark the start vertex as visited and enqueue it
    visited[start] = 1;
    queue[rear++] = start;

```

```

printf("BFS starting from vertex %d:\n", start);

while (front < rear) {
    int vertex = queue[front++]; // Dequeue a vertex
    printf("%d ", vertex);

    // Get all adjacent vertices of the dequeued vertex
    struct Node* temp = graph->adjList[vertex];
    while (temp != NULL) {
        int adjVertex = temp->vertex;
        if (!visited[adjVertex]) {
            visited[adjVertex] = 1; // Mark the adjacent vertex as visited
            queue[rear++] = adjVertex; // Enqueue the adjacent vertex
        }
        temp = temp->next;
    }
}
printf("\n");
}

int main() {
    struct Graph graph;
    int n, e, u, v;

    // Take the number of vertices and edges as input
    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the number of edges: ");

```

```

scanf("%d", &e);

// Initialize the graph
initGraph(&graph, n);

// Input edges
printf("Enter the edges (u v) where u and v are vertices connected by an edge:\n");
for (int i = 0; i < e; i++) {
    scanf("%d %d", &u, &v);
    addEdge(&graph, u, v);
}

// Perform BFS starting from vertex 0 (or any vertex)
bfs(&graph, 0, n);

return 0;
}

```

OUTPUT:-

Output

Clear

```

Enter the number of vertices: 6
Enter the number of edges: 6
Enter the edges (u v) where u and v are vertices connected by an edge:
6
0 1
0 2
1 3
1 4
2 4
3 5
BFS starting from vertex 0:
0 1 6 3 2 4

=== Code Execution Successful ===

```

PROGRAM

18 Write a C program to demonstrate basic file operations : open, read, write and close.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    FILE *file;
```

```
    char filename[] = "example.txt";
```

```
    char buffer[100];
```

```
    // Writing to a file
```

```
    file = fopen(filename, "w"); // Open file for writing ("w" mode)
```

```
    if (file == NULL) {
```

```
        perror("Error opening file for writing");
```

```
        return 1;
```

```
    }
```

```
    fprintf(file, "This is an example of file operations in C.\n");
```

```
    fprintf(file, "We are demonstrating open, write, read, and close operations.\n");
```

```
    printf("Data written to file successfully.\n");
```

```
    // Close the file after writing
```

```
    fclose(file);
```

```
    // Reading from a file
```

```
    file = fopen(filename, "r"); // Open file for reading ("r" mode)
```

```
    if (file == NULL) {
```

```
        perror("Error opening file for reading");
        return 1;
    }

    printf("\nReading data from file:\n");

    // Read the file line by line and print it
    while (fgets(buffer, sizeof(buffer), file)) {
        printf("%s", buffer);
    }

    // Close the file after reading
    fclose(file);

    return 0;
}
```

OUTPUT:-

A screenshot of a terminal window with a dark background. The title bar at the top says "Output" in orange text on the left and "Clear" in white text on the right, enclosed in a button. The terminal content shows the message "Error opening file for writing: Permission denied" in white text. Below this, there is a separator line consisting of three equals signs, followed by the text "Code Exited With Errors" in a lighter gray font, also flanked by three equals signs.

```
Output Clear
Error opening file for writing: Permission denied

=== Code Exited With Errors ===
```